Práctica 2A

Aplanar un mamut

Aprendizaje No Supervisado y Visualización

Unidad 2 · Maestría en Ciencia de Datos e Información

Bryan Rodríguez Abarca

Matrícula 26DF1185A9

17 de septiembre de 2026

INFOTEC · Ciudad de México

El mamut son 10000 puntos con tres coordenadas cada uno: la superficie de un animal digitalizada. Sobre eso hay que correr k-means con k = 8, verlo en 3D con un color por grupo, aplanarlo a 2D con PCA, t-SNE y UMAP, y decidir cuál de las tres versiones planas sirve más.

```
import platform, time
import numpy as np
import sklearn, umap
import matplotlib
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.neighbors import NearestNeighbors, kneighbors_graph
from scipy.sparse.csgraph import connected_components
from scipy.stats import pearsonr

print("python      ", platform.python_version())
print("numpy       ", np.__version__)
print("scikit-learn", sklearn.__version__)
print("umap-learn  ", umap.__version__)
print("matplotlib  ", matplotlib.__version__)
```

```
python       3.11.15
numpy        2.4.6
scikit-learn 1.9.0
umap-learn   0.5.12
matplotlib   3.11.1
```

1. Los datos

Miré los rangos de las tres columnas antes de agrupar, porque k-means usa distancia euclidiana y una coordenada en otra escala se comería a las demás.

```
X = np.load("mamut_datos.npy")
print("forma:", X.shape, X.dtype)
for i, nombre in enumerate("xyz"):
    c = X[:, i]
    print(nombre, "min", round(c.min(), 2), "max", round(c.max(), 2),
          "rango", round(c.max() - c.min(), 2))
```

```
forma: (10000, 3) float64
x min -21.68 max 168.93 rango 190.61
y min 61.37 max 568.67 rango 507.3
z min -171.92 max 171.69 rango 343.61
```

La segunda coordenada abarca 507 unidades y la primera 191, así que `y` es el largo del animal y pesa casi el triple que `x` en cualquier distancia. No reescalo: las tres columnas son posición, la misma magnitud física, y estirarlas para igualar rangos deformaría la geometría que quiero agrupar.

2. k-means con k = 8

La primera corrida la hice sin tocar `n_init` y las patas traseras salieron partidas a media altura, la mitad de arriba de un color y la de abajo de otro. Volví a la gráfica 3D a leer entre qué coordenadas cae esa zona para poder contar los puntos, y corrí de nuevo con `n_init=10`, que repite el arranque diez veces y se queda con el mejor.

```
km = KMeans(n_clusters=8, random_state=0)
km.fit(X)
km10 = KMeans(n_clusters=8, random_state=0, n_init=10)
km10.fit(X)
print("inercia por omision:", round(km.inertia_))
print("inercia con n_init=10:", round(km10.inertia_))
print("rand ajustado entre las dos particiones:",
      round(adjusted_rand_score(km.labels_, km10.labels_), 4))

# zona de patas traseras, leida de la grafica 3D
zona = (X[:, 1] > 450) & (X[:, 2] < -60)
print("puntos en la zona:", zona.sum())
print("por omision se reparten en:", np.bincount(km.labels_[zona]))
print("con n_init=10:", np.bincount(km10.labels_[zona]))

etq = km10.labels_
tam = np.bincount(etq)
for c in range(8):
    print("grupo", c, tam[c], "puntos, centro",
          np.round(km10.cluster_centers_[c], 1))
```

```
inercia por omision: 29585020
inercia con n_init=10: 26320742
rand ajustado entre las dos particiones: 0.7693
puntos en la zona: 844
por omision se reparten en: [  0   0   0 286   0   0   0 558]
con n_init=10: [  0 844]
grupo 0 1218 puntos, centro [ 70.  317.1   6.7]
grupo 1 924 puntos, centro [  76.9  512.2 -118.4]
grupo 2 440 puntos, centro [ 67.4 123.1  53.3]
grupo 3 834 puntos, centro [  75.2  300.8 -133. ]
grupo 4 1324 puntos, centro [ 78.8 236.  110. ]
grupo 5 1808 puntos, centro [ 76.3 411.7 112.9]
grupo 6 2270 puntos, centro [ 80.6 323.6  99.5]
grupo 7 1182 puntos, centro [ 76.9 506.5  49.3]
```

Los 844 puntos de esa zona quedaban repartidos 286 / 558 entre dos grupos y con `n_init=10` pasaron a uno solo. La inercia bajó de 29585020 a 26320742, un 11 % menos, y el índice de Rand ajustado entre las dos particiones es 0.7693, o sea que entre las dos hay puntos que cambiaron de grupo, no una renumeración. Uso `km10` de aquí en adelante. `random_state=0` va para que el color de cada parte no cambie entre corridas.

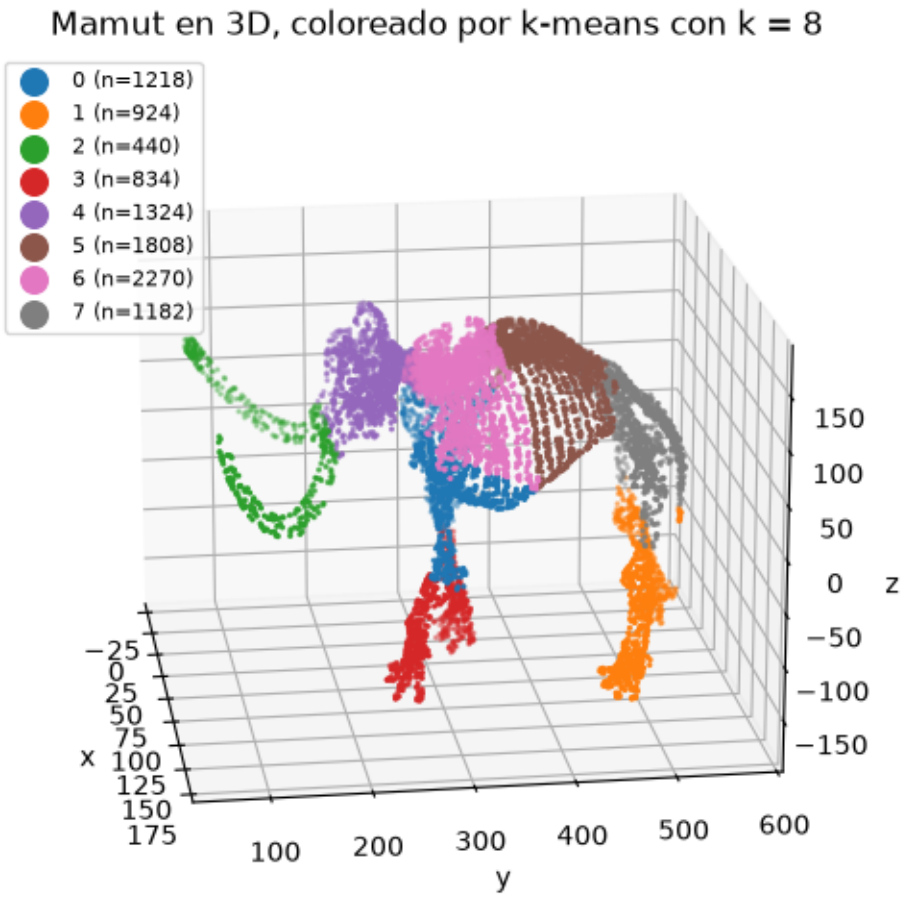
Los tamaños son desparejos, de 440 a 2270 puntos. k-means no reparte por cantidad sino por cercanía, y una parte del cuerpo que ocupa poco volumen se lleva pocos puntos.

3. La gráfica 3D

El ángulo por omisión mira al animal en diagonal desde arriba y encima unas patas con otras; con `elev=18, azim=-8`, casi de perfil, se separan y además quedan los colmillos por delante. Dibujo grupo por grupo en vez de pasar `c=etq` para poder poner leyenda.

```
COLOR = [plt.get_cmap("tab10")(c) for c in range(8)]

fig = plt.figure(figsize=(7.5, 6))
ax = fig.add_subplot(projection="3d")
for c in range(8):
    m = etq == c
    ax.scatter(X[m, 0], X[m, 1], X[m, 2], s=1.5,
               color=COLOR[c], label=f"{c} (n={tam[c]})")
ax.view_init(elev=18, azim=-8)
ax.set_xlabel("x"); ax.set_ylabel("y"); ax.set_zlabel("z")
ax.set_title("Mamut en 3D, coloreado por k-means con k = 8")
ax.legend(markerscale=8, fontsize=8, loc="upper left")
plt.show()
```



4. Qué parte del mamut es cada grupo

Los ocho grupos se leen como partes del animal. De adelante hacia atrás: el 2 son los colmillos, los dos arcos que salen hacia el frente; el 4 es la cabeza; el 6 es la cruz, el bloque que va del cuello al arranque del lomo; el 5 es el lomo; el 7 es la grupa con la cola; el 0 es el vientre y la parte alta de las patas delanteras; el 3 son las patas delanteras y el 1 las traseras.

En la cola el color cambia justo antes de que la cola termine, y eso sí lo fui a medir.

```
cola = (X[:, 1] > 540) & (X[:, 2] > -60)
print("puntos de la cola:", cola.sum())
print("repartidos en:", np.bincount(etq[cola]))
for c in [1, 7]:
    z = X[cola & (etq == c)][:, 2]
    print("  los del grupo", c, "van de z =", round(z.min(), 1),
          "a z =", round(z.max(), 1))
print("centro en z del grupo 1:", round(X[etq == 1][:, 2].mean(), 1))
print("centro en z del grupo 7:", round(X[etq == 7][:, 2].mean(), 1))
```

```
puntos de la cola: 156
```

```
repartidos en: [  0  12   0   0   0   0   0 144]
  los del grupo 1 van de z = -46.6 a z = -34.5
  los del grupo 7 van de z = -30.3 a z = 52.9
centro en z del grupo 1: -118.4
centro en z del grupo 7: 49.4
```

De los 156 puntos de la cola, 144 son del grupo 7 y 12 del grupo 1. Los 12 son los de más abajo, de $z = -46.6$ a $z = -34.5$, debajo de los del grupo 7, que llegan hasta -30.3. A esa altura el centro del grupo 1 —$z = -118.4$, con las patas traseras— les queda más cerca que el del grupo 7, que está en $z = 49.4$, arriba en la grupa. k-means cortó la cola a media altura y le dio la punta a las patas traseras.

5. Las tres proyecciones a 2D

Con los nombres de las partes ya puedo poner una leyenda legible. Las tres proyecciones salen de los mismos `X` y los mismos colores, así que lo único que cambia entre ellas es el método.

```python
PARTES = {0: "vientre", 1: "patas traseras", 2: "colmillos",
          3: "patas delanteras", 4: "cabeza", 5: "lomo",
          6: "cruz", 7: "grupa y cola"}

pca3 = PCA(n_components=3, random_state=0).fit(X)
print("varianza explicada:", np.round(pca3.explained_variance_ratio_, 4))
print("las dos primeras suman",
      round(pca3.explained_variance_ratio_[:2].sum(), 4))

t = time.time()
Zp = PCA(n_components=2, random_state=0).fit_transform(X)
print("PCA  ", round(time.time() - t, 2), "s")

t = time.time()
# con init="pca" la corrida sale igual cada vez
Zt = TSNE(n_components=2, random_state=0, init="pca").fit_transform(X)
print("t-SNE", round(time.time() - t, 2), "s")

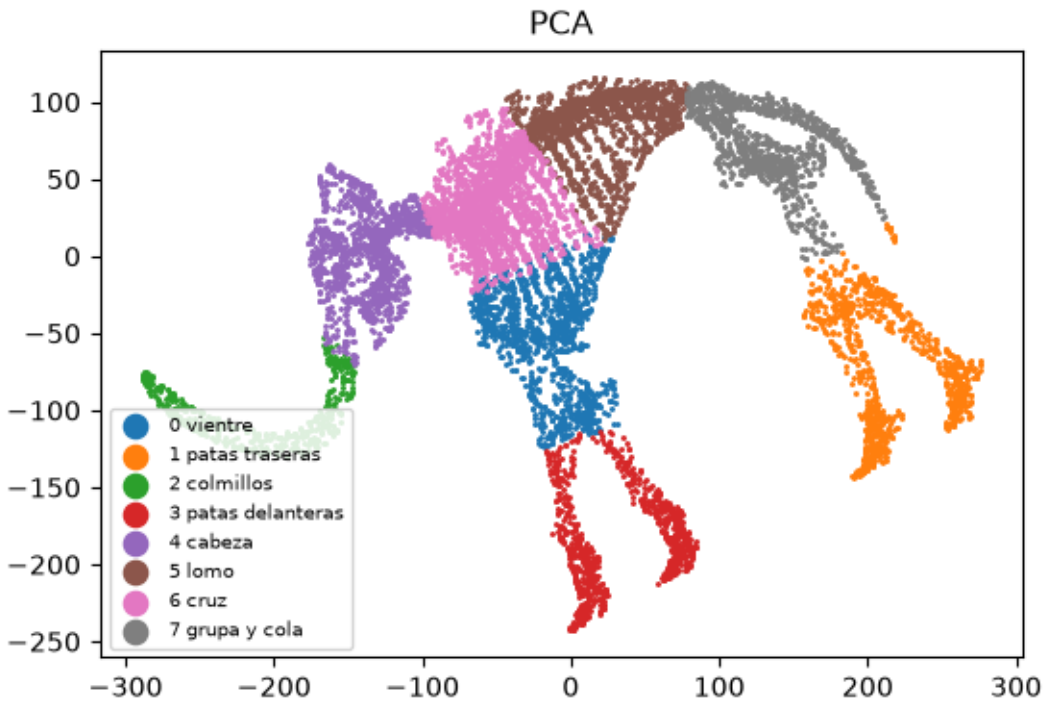
t = time.time()
Zu = umap.UMAP(n_components=2, random_state=0,
               n_jobs=1).fit_transform(X)   # n_jobs=1 evita el warning
print("UMAP ", round(time.time() - t, 2), "s")
```

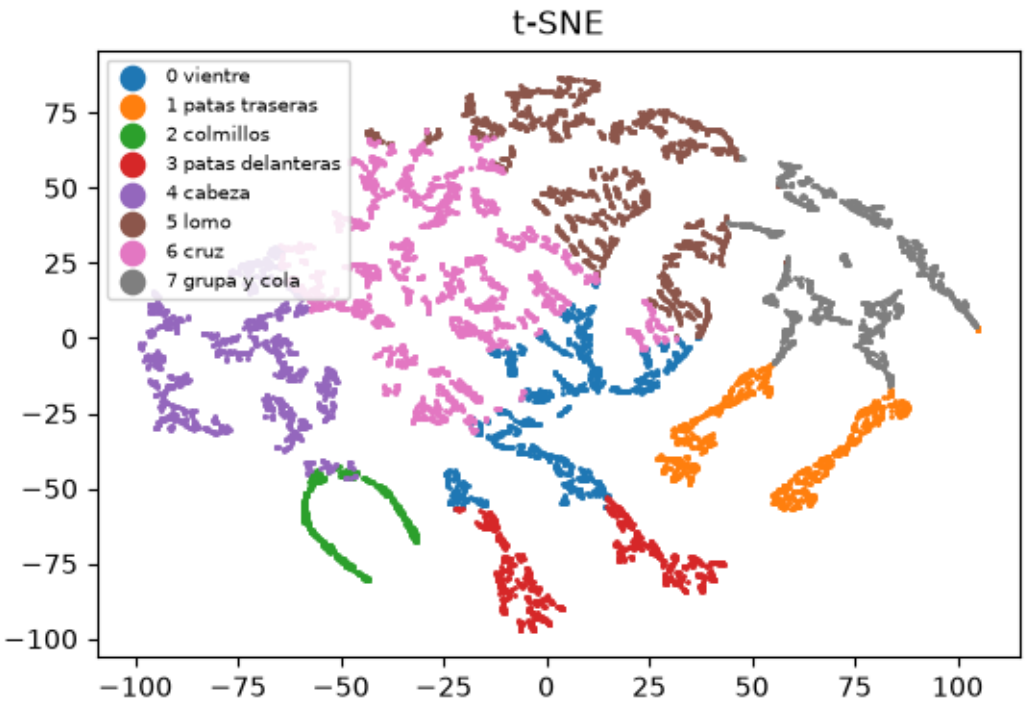
```
varianza explicada: [0.6007 0.3425 0.0568]
las dos primeras suman 0.9432
PCA   0.0 s
t-SNE 25.05 s
UMAP  20.35 s
```

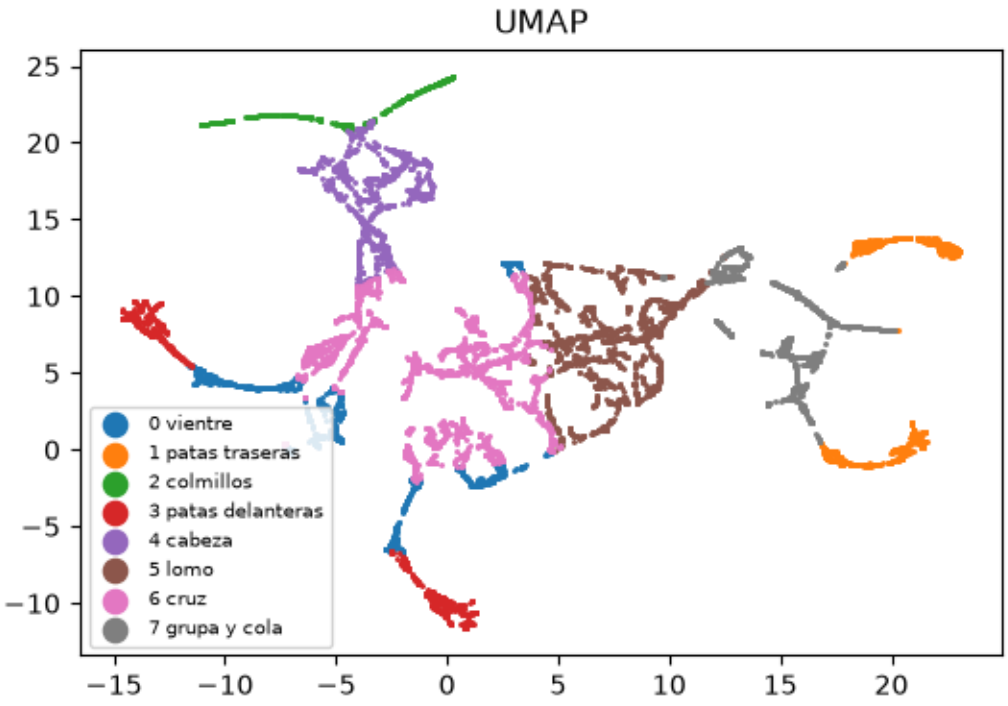
La primera componente se lleva el 60.07 % de la varianza, la segunda el 34.25 % y la tercera solo el 5.68 %. Lo que PCA tira es el ancho del animal, que es su dimensión más corta, y por eso la proyección sale de perfil.

```
def dibuja(Z, titulo):
    fig, ax = plt.subplots(figsize=(6.2, 4.1))
    for c in range(8):
        m = etq == c
        ax.scatter(Z[m, 0], Z[m, 1], s=1.2, color=COLOR[c],
                   label=f"{c} {PARTES[c]}")
    ax.set_title(titulo)
    ax.legend(markerscale=8, fontsize=7, loc="best", framealpha=0.85)
    plt.show()


dibuja(Zp, "PCA")
dibuja(Zt, "t-SNE")
dibuja(Zu, "UMAP")
```







PCA devuelve un mamut de perfil que se reconoce sin leyenda. t-SNE y UMAP devuelven otra cosa: cada parte sigue siendo una mancha de su color, pero las manchas están picadas de huecos blancos y colocadas en un orden que ya no es el del animal. Dónde exactamente se descolocan lo mido más abajo, porque a ojo me estaba inventando lecturas.

6. Cómo elijo

Las tres me parecían defendibles por motivos distintos, así que en lugar de quedarme con la que se ve mejor armé cuatro números. Nada de esto lo pide el enunciado; es trabajo mío para poder defender la elección.

Junto a cada número pongo un piso: una proyección lineal a 2D con una matriz de 3×2 sacada al azar. Aquí solo se tira una dimensión de tres, así que sospechaba que un plano cualquiera ya conserva bastante, y sin ese piso no sabría si una cifra alta es mérito del método o del problema.

6.1. Vecinos que sobreviven al aplanado

```
def vecinos(A, B, k):
    a = NearestNeighbors(n_neighbors=k + 1).fit(A)
    b = NearestNeighbors(n_neighbors=k + 1).fit(B)
    va = a.kneighbors(return_distance=False)[:, :k]
    vb = b.kneighbors(return_distance=False)[:, :k]
    comunes = [len(set(u) & set(v)) for u, v in zip(va, vb)]
    return np.mean(comunes) / k


rng = np.random.default_rng(0)
Za = X @ rng.normal(size=(3, 2))
proys = [("PCA", Zp), ("t-SNE", Zt), ("UMAP", Zu), ("azar", Za)]

print("k, PCA, t-SNE, UMAP, azar")
for k in [5, 10, 20, 50]:
    print(k, [round(float(vecinos(X, Z, k)), 4) for _, Z in proys])
```

```
k, PCA, t-SNE, UMAP, azar
5 [0.3937, 0.7853, 0.6323, 0.3862]
10 [0.4143, 0.7929, 0.6871, 0.4045]
20 [0.4428, 0.7842, 0.7067, 0.4322]
50 [0.4982, 0.7713, 0.7219, 0.4821]
```

Tomo los k vecinos más cercanos de cada punto en 3D y los k del mismo punto en la proyección, y cuento cuántos se repiten. t-SNE gana en los cuatro tamaños: 0.7842 contra 0.4428 de PCA con k = 20. UMAP queda en 0.7067, y el plano al azar saca 0.4322, prácticamente lo mismo que PCA. En vecindad local el plano de máxima varianza no le gana a un plano cualquiera, porque ninguna dirección de tres aplana mucho más que otra; lo que t-SNE y UMAP hacen de más es mover los puntos, no solo mirarlos desde otro lado.

6.2. Qué tan separadas quedan las partes en el plano

La silueta la calculo sobre las coordenadas 2D pero con las etiquetas del k-means de 3D, así que no premia a un agrupamiento sino a la proyección que mantiene apartados los grupos que ya existían.

Reuso el control de la práctica anterior: reasigno las etiquetas al azar dejando intactos los ocho tamaños. Como un solo plano al azar puede haber salido bueno o malo por suerte, corro veinte.

```
espacios = [("3D", X), ("PCA", Zp), ("t-SNE", Zt), ("UMAP", Zu),
            ("azar", Za)]
barajadas = np.random.default_rng(0).permutation(etq)

print("espacio, silueta, control barajado")
for n, A in espacios:
    print(n, round(silhouette_score(A, etq), 4),
          round(silhouette_score(A, barajadas), 4))

r20 = np.random.default_rng(1)
sil20 = [silhouette_score(X @ r20.normal(size=(3, 2)), etq)
         for _ in range(20)]
print("20 planos al azar: media", round(np.mean(sil20), 4),
      "min", round(min(sil20), 4), "max", round(max(sil20), 4))
```

```
espacio, silueta, control barajado
3D 0.3815 -0.0185
PCA 0.4813 -0.0194
t-SNE 0.3081 -0.0184
UMAP 0.252 -0.0201
azar 0.425 -0.0236
20 planos al azar: media 0.148 min -0.1402 max 0.4166
```

Aquí el orden se invierte. PCA saca 0.4813, por encima del 0.3815 que dan los datos en 3D y de t-SNE (0.3081) y UMAP (0.2520). Los cinco controles barajados caen entre -0.0236 y -0.0184: el reparto de tamaños por sí solo no produce silueta.

El piso al azar es lo que le da peso al resultado. Los veinte planos dan una media de 0.148 y el mejor de ellos llega a 0.4166, todavía por debajo del 0.4813 de PCA.

Que PCA supere al propio 3D no es un error: al tirar el ancho del animal los grupos se vuelven más planos sin acercarse entre sí, porque están separados a lo largo del cuerpo y no a lo ancho.

6.3. Si cada parte llega entera

Al ver los huecos blancos de t-SNE y UMAP quise contarlos sin adjetivos. Buscando cómo contar pedazos en un grafo de vecinos encontré que scipy trae `connected_components`, así que dentro de cada grupo por separado armo el grafo de los 10 vecinos más cercanos y cuento sus componentes.

De paso volví al 3D, porque antes de acusar a una proyección de partir algo hay que saber qué venía ya partido. Los grupos de patas los sospechaba: si k-means separó adelante de atrás en lugar de izquierda de derecha, cada uno tiene que ser un par de patas.

```python
def piezas(A, k=10):
    out = []
    for c in range(8):
        P = A[etq == c]
        n, _ = connected_components(kneighbors_graph(P, k), directed=False)
        out.append(n)
    return out


print("espacio, piezas por grupo, total")
for n, A in espacios:
    p = piezas(A)
    print(n, p, sum(p))

print("totales con otros k:")
for k in [5, 20]:
    print(" k =", k, [(n, sum(piezas(A, k))) for n, A in espacios])

for c in [1, 2, 3]:
    lado = X[etq == c][:, 0] < 75
    print("grupo", c, "se parte", lado.sum(), "/", (~lado).sum(),
          "a los lados de x = 75")
```

```
espacio, piezas por grupo, total
3D [1, 3, 1, 2, 1, 1, 1, 1] 11
PCA [1, 2, 1, 1, 1, 1, 1, 1] 9
t-SNE [9, 4, 3, 3, 12, 15, 29, 9] 84
UMAP [8, 4, 3, 6, 3, 10, 13, 6] 53
azar [1, 2, 1, 2, 1, 1, 1, 1] 10
totales con otros k:
 k = 5 [('3D', 18), ('PCA', 10), ('t-SNE', 217), ('UMAP', 183), ('azar', 13)]
 k = 20 [('3D', 10), ('PCA', 8), ('t-SNE', 33), ('UMAP', 22), ('azar', 8)]
grupo 1 se parte 461 / 463 a los lados de x = 75
grupo 2 se parte 248 / 192 a los lados de x = 75
grupo 3 se parte 409 / 425 a los lados de x = 75
```

En 3D los ocho grupos dan 11 piezas: seis enteros, el 3 partido en dos —las dos patas delanteras, 409 y 425 puntos— y el 1 en tres, las dos traseras más los 12 puntos de la punta de la cola. Los colmillos se parten 248 / 192 a los lados pero cuentan como una sola pieza, porque los dos arcos están unidos por la base; ese 11 no los cuenta como dos.

t-SNE da 84 y UMAP 53. El lomo, una sola pieza de 1808 puntos en el animal, sale en 15 pedazos con t-SNE y en 10 con UMAP; la cruz, la más grande con 2270 puntos, en 29 y 13. Con k = 5 y k = 20 los totales cambian pero la distancia entre esos dos y el resto se mantiene.

Lo que esta medición no puede hacer es coronar a PCA. Su 9 y el 10 del plano al azar están dentro del mismo margen, así que la cuenta de piezas separa a las proyecciones que despedazan de las que no, y entre las que no, no ordena. La retiro como argumento a favor de PCA.

Sobre ese 9: no es que PCA encime una pata sobre otra. Eso lo di por hecho al principio y es falso.

```
for n, A in [("3D", X), ("PCA", Zp)]:
    for c in [1, 3]:
        m = etq == c
        lado = X[m][:, 0] < 75
        P = A[m]
        d = np.linalg.norm(P[lado].mean(0) - P[~lado].mean(0))
        v = NearestNeighbors(n_neighbors=11).fit(P).kneighbors(
            return_distance=False)[:, :10]
        cruz = (lado[v] != lado[:, None]).mean()
        print(n, "grupo", c, "| centros de las dos patas a", round(d, 1),
              "| vecinos en la pata contraria", round(100 * cruz, 3), "%")

for c in [1, 3]:
    P = X[etq == c]
    lado = P[:, 0] < 75
    print("grupo", c, "y medio de cada pata:", round(P[lado][:, 1].mean(), 1),
          "y", round(P[~lado][:, 1].mean(), 1))
```

```
3D grupo 1 | centros de las dos patas a 88.0 | vecinos en la pata contraria 0.411 %
3D grupo 3 | centros de las dos patas a 81.1 | vecinos en la pata contraria 0.0 %
PCA grupo 1 | centros de las dos patas a 49.7 | vecinos en la pata contraria 2.273 %
PCA grupo 3 | centros de las dos patas a 60.9 | vecinos en la pata contraria 0.144 %
grupo 1 y medio de cada pata: 536.7 y 487.8
grupo 3 y medio de cada pata: 331.4 y 271.2
```

En el plano de PCA los centros de las dos patas delanteras quedan a 60.9 unidades y solo el 0.144 % de los vecinos de un punto caen en la pata de enfrente; en las traseras, 49.7 unidades y 2.273 %. Las patas no se enciman: el mamut está en zancada, con sus patas delanteras centradas en y = 331.4 y y = 271.2, y esa separación sobrevive al aplanado. Lo que pasa es que basta un puñado de puntos en contacto para que el grafo pegue las dos piezas, y ese puñado existe. La cuenta de piezas es sensible a una costura de dos puntos.

6.4. La geometría entre las partes

Queda por medir si las partes conservan su acomodo. Tomo el centro de cada grupo en 3D y en la proyección, calculo las 28 distancias entre pares de centros de cada lado y las correlaciono.

```
cen3 = np.array([X[etq == c].mean(0) for c in range(8)])
print("los 8 centros abarcan en x", round(cen3[:, 0].min(), 1), "a",
      round(cen3[:, 0].max(), 1), "; en y", round(cen3[:, 1].min(), 1),
      "a", round(cen3[:, 1].max(), 1))

arriba = np.triu_indices(8, 1)
d3 = np.linalg.norm(cen3[:, None] - cen3[None], axis=2)[arriba]
```

```
def corr(A):
    c2 = np.array([A[etq == c].mean(0) for c in range(8)])
    d2 = np.linalg.norm(c2[:, None] - c2[None], axis=2)[arriba]
    return pearsonr(d3, d2)[0]


for n, Z in proys:
    print(n, round(corr(Z), 6))

r20b = np.random.default_rng(1)
c20 = [corr(X @ r20b.normal(size=(3, 2))) for _ in range(20)]
print("20 planos al azar: media", round(np.mean(c20), 4),
      "min", round(min(c20), 4), "max", round(max(c20), 4))

ct = np.array([Zt[etq == c].mean(0) for c in range(8)])
dt = np.linalg.norm(ct[:, None] - ct[None], axis=2)[arriba]
resto = dt / dt.max() - d3 / d3.max()
for j in [np.argmin(resto), np.argmax(resto)]:
    a, b = arriba[0][j], arriba[1][j]
    print("t-SNE:", PARTES[a], "y", PARTES[b], "cambian", round(resto[j], 3))
```

```
los 8 centros abarcan en x 67.3 a 80.6 ; en y 122.8 a 512.2
PCA 0.999999
t-SNE 0.741774
UMAP 0.76055
azar 0.890413
20 planos al azar: media 0.718 min 0.3246 max 0.9697
t-SNE: patas traseras y colmillos cambian -0.291
t-SNE: cabeza y grupa y cola cambian 0.348
```

PCA da 0.999999. La razón está en los propios centros: los ocho abarcan 13.3 unidades en `x` y 389.4 en `y`, o sea que caen casi en un mismo plano, y ese plano es justo el que PCA conserva.

t-SNE saca 0.7418 y UMAP 0.7606, y los veinte planos al azar promedian 0.718 con un mejor caso de 0.9697. t-SNE y UMAP caen dentro de ese rango; PCA queda arriba de los veinte. La distorsión mayor de t-SNE es que acerca los colmillos a las patas traseras, 0.291 de distancia relativa, y aleja la cabeza de la grupa, 0.348.

7. Un barrido de parámetros

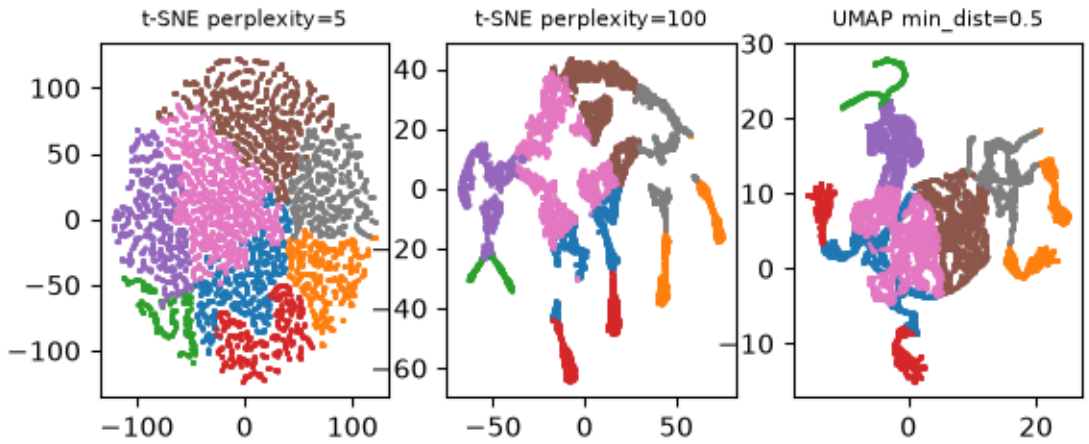
Antes de cerrar quise ver cuánto de esto es culpa de los valores por omisión. De t-SNE muevo `perplexity` y también `init`, que con `"pca"` hace la corrida repetible pero no es lo que t-SNE hace si se le deja al azar. De UMAP muevo `min_dist`, que regula qué tan apretados quedan los puntos, y `n_neighbors`.

```
def mide(Z, nombre):
    print(nombre, "| vecinos(k=20)", round(vecinos(X, Z, 20), 4),
          "| silueta", round(silhouette_score(Z, etq), 4),
          "| piezas", sum(piezas(Z)))


fig, axes = plt.subplots(1, 3, figsize=(6.6, 2.4))
corridas = [
    ("t-SNE perplexity=5", TSNE(n_components=2, random_state=0,
                                init="pca", perplexity=5)),
    ("t-SNE perplexity=100", TSNE(n_components=2, random_state=0,
                                  init="pca", perplexity=100)),
    ("UMAP min_dist=0.5", umap.UMAP(n_components=2, random_state=0,
                                    n_jobs=1, min_dist=0.5)),
]
for ax, (nombre, modelo) in zip(axes, corridas):
    Z = modelo.fit_transform(X)
    mide(Z, nombre)
    for c in range(8):
        m = etq == c
        ax.scatter(Z[m, 0], Z[m, 1], s=0.6, color=COLOR[c])
    ax.set_title(nombre, fontsize=8)
plt.show()

mide(TSNE(n_components=2, random_state=0,
          init="random").fit_transform(X), "t-SNE init=random")
mide(umap.UMAP(n_components=2, random_state=0, n_jobs=1,
               n_neighbors=50).fit_transform(X), "UMAP n_neighbors=50")
```

```
t-SNE perplexity=5 | vecinos(k=20) 0.6946 | silueta 0.2439 | piezas 110
t-SNE perplexity=100 | vecinos(k=20) 0.7716 | silueta 0.3178 | piezas 23
UMAP min_dist=0.5 | vecinos(k=20) 0.7186 | silueta 0.2498 | piezas 13
```



```
t-SNE init=random | vecinos(k=20) 0.7825 | silueta 0.1825 | piezas 74
UMAP n_neighbors=50 | vecinos(k=20) 0.6738 | silueta 0.3198 | piezas 21
```

Mover los parámetros cambia bastante. `min_dist=0.5` le baja a UMAP las piezas de 53 a 13, o sea que buena parte del picado era el valor por omisión y no el método. `n_neighbors=50` le sube la silueta a 0.3198, por encima del 0.3081 de t-SNE. En t-SNE, `perplexity` a 5 baja la silueta a 0.2439 y a 100 la sube a 0.3178, y cambiar `init` a `"random"` la derrumba a 0.1825. Esa caída de 0.126 es más grande que los 0.056 que separan a t-SNE de UMAP con los valores por omisión, así que no tengo base para ordenar esas dos entre sí. De las cinco corridas, ninguna llega a la silueta de PCA.

8. La proyección que elijo

PCA. Dos mediciones la sostienen y las dos aguantan el piso al azar. La silueta de los ocho grupos medida sobre el plano da 0.4813, contra 0.3081 de t-SNE, 0.2520 de UMAP y 0.4166 del mejor de veinte planos al azar; y la correlación entre las distancias de las partes en 3D y en el plano da 0.999999, contra 0.7418, 0.7606 y un mejor caso al azar de 0.9697. Las partes quedan separadas entre sí y colocadas donde estaban, que es lo que la práctica le pide a la figura.

Lo que no puedo alegar es que conserve la vecindad: con k = 20 sobrevive el 44.28 % de los vecinos contra el 78.42 % de t-SNE, y un plano al azar saca 43.22 %. Ahí PCA está en el piso. Tampoco me apoyo en la cuenta de piezas: sus 9 no se distinguen de las 10 del plano al azar, y de todos modos salen de pegar cada par de patas por una costura de pocos puntos.

El veredicto vale para las tres con sus valores por omisión. UMAP con `n_neighbors=50` supera a t-SNE en silueta y con `min_dist=0.5` deja de despedazar; ninguna de las dos alcanza a PCA, pero el orden entre t-SNE y UMAP depende de cómo se les configure.

9. Conclusión

k-means con k = 8, `random_state=0` y `n_init=10` corta al mamut en partes que coinciden con su anatomía: colmillos con 440 puntos, cabeza 1324, cruz 2270, lomo 1808, grupa y cola 1182, vientre 1218, patas delanteras 834 y traseras 924. El único corte que no sigue al animal es la punta de la cola, cuyos 12 puntos más bajos se van con las patas traseras. Los grupos de patas son pares completos: 409 / 425 y 461 / 463 a los lados de x = 75.

Elijo PCA por la silueta (0.4813) y por la correlación entre las distancias de las partes (0.999999), que son las dos mediciones en las que queda fuera del alcance de un plano al azar. Conserva el 94.32 % de la varianza y tira el ancho del animal, que es donde los ocho centros apenas se extienden 13.3 unidades; por eso el acomodo de las partes sobrevive casi exacto.

El piso al azar salió alto. Un plano al azar conserva el 43.22 % de los vecinos, deja los grupos en 10 piezas y correlaciona las distancias entre partes a 0.718 de media: con tres dimensiones de origen, buena parte de lo que parecía mérito de un método lo consigue tirar una coordenada cualquiera. De mis cuatro mediciones, dos no distinguen a PCA de ese plano.